

Middleware

Обеспеч. проверку и фильтрацию HTTP-запросов до или после обработки в приложении.

```
php artisan make:middleware EnsureTokenIsValid
```

```
// тело класса мидвара
public function handle($request, Closure $next, $role)
{
    if (! $request->user()->hasRole($role)) {
        // Redirect...
    }

    // Выполнение действий до обработки запроса
    // Действия ...
    return $next($request);

    // Выполнение действий после обработки запроса
    $response = $next($request);
    // Действия ...
    return $response;
}

// Автоматически вызывается после отправки ответа в браузер.
// Для вызова тем же объектом, что и для handle()
// использ. $this->app->singleton() в App\Providers\AppServiceProvider::register()
public function terminate($request, $response)
{
    // ...
}
```

```
// Регистрация мидвара
```

```
// class App\Http\Kernel
protected $middleware = [/** */]; // глобальные
protected $middlewareGroups = [/** */]; // группы мидлваров
protected $routeMiddleware = [/** */]; // для роутов
protected $middlewarePriority = [/** */]; // свой порядок выполнения мидлваров
```

```
// использование мидлвара
```

```
Route::get('/** */->middleware(['first', 'second']));
```

```
Route::get('/** */->middleware(EnsureTokenIsValid::class);
```

```
Route::middleware([EnsureTokenIsValid::class])>group(function () {
```

```
    // ...
```

```
    Route::get('/** */->withoutMiddleware([EnsureTokenIsValid::class]);
```

```
});
```

```
Route::withoutMiddleware([EnsureTokenIsValid::class])>group(function () {
```

```
    Route::get('/profile', function () {/** */});
```

```
});
```

```
// использовать один и тот же экземпляр мидлвара при вызове handle() и terminate()
```

```
// class App\Providers\AppServiceProvider
```

```
public function register()
```

```
{
```

```
    $this->app->singleton(TerminatingMiddleware::class);
```

```
}
```